

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: ITA0605**

**COURSE NAME: MACHINE LEARNING**

### **COURSE OUTCOME COVERED**

**CO1:** *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

*Assessment Tool Weightage: 40%*

**QUESTIONS: 15**

**Total Marks: 25**

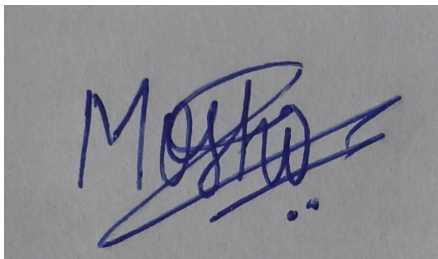
Annexure A

### **APPLICATION BASED QUESTIONS**

#### ***Code of Conduct:***

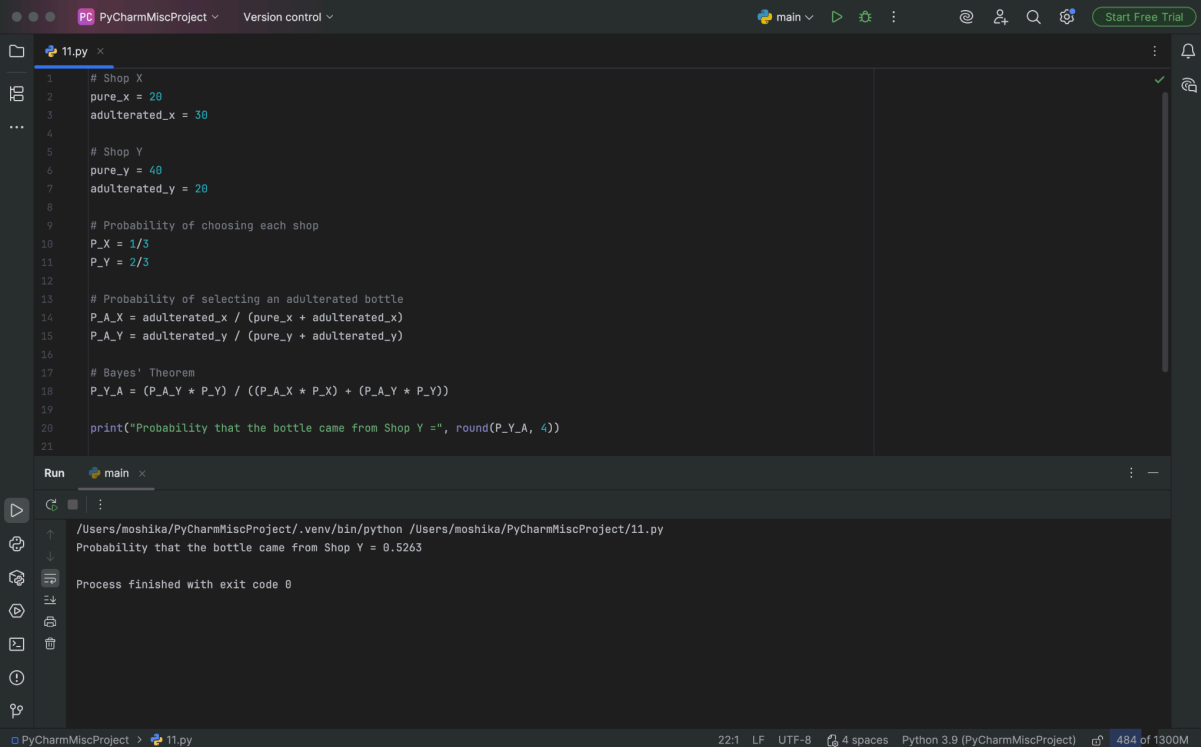
*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

A handwritten signature in blue ink, appearing to read 'Moshika', is written on a light-colored background. The signature is stylized with a large, sweeping underline.

11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated. (5 Marks)

ANSWER:



```
1 # Shop X
2 pure_x = 20
3 adulterated_x = 30
4
5 # Shop Y
6 pure_y = 40
7 adulterated_y = 20
8
9 # Probability of choosing each shop
10 P_X = 1/3
11 P_Y = 2/3
12
13 # Probability of selecting an adulterated bottle
14 P_A_X = adulterated_x / (pure_x + adulterated_x)
15 P_A_Y = adulterated_y / (pure_y + adulterated_y)
16
17 # Bayes' Theorem
18 P_Y_A = (P_A_Y * P_Y) / ((P_A_X * P_X) + (P_A_Y * P_Y))
19
20 print("Probability that the bottle came from Shop Y =", round(P_Y_A, 4))
21
```

Run main ×

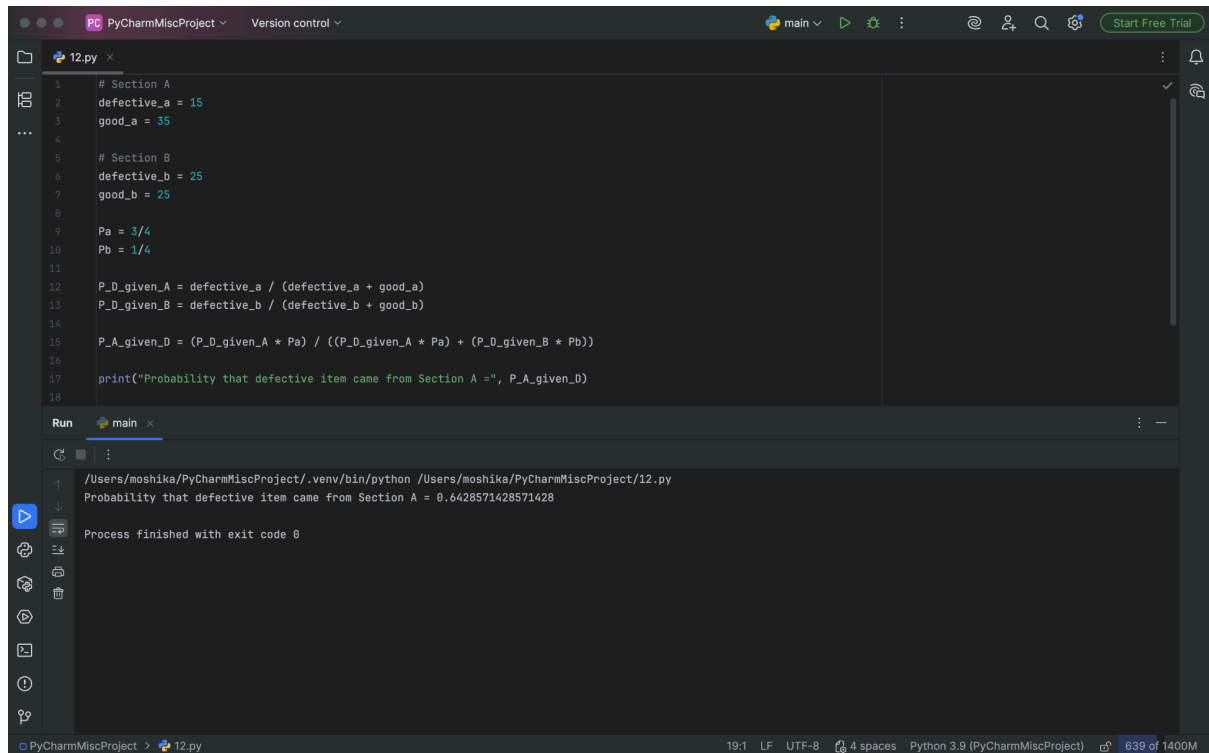
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/11.py  
Probability that the bottle came from Shop Y = 0.5263

Process finished with exit code 0

PyCharmMiscProject > 11.py 22:1 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 484 of 1300M

**12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective. (5 Marks)**

**ANSWER:**



The screenshot shows a PyCharm IDE window for a project named 'PyCharmMiscProject'. The editor displays a Python file '12.py' with the following code:

```
1 # Section A
2 defective_a = 15
3 good_a = 35
4
5 # Section B
6 defective_b = 25
7 good_b = 25
8
9 Pa = 3/4
10 Pb = 1/4
11
12 P_D_given_A = defective_a / (defective_a + good_a)
13 P_D_given_B = defective_b / (defective_b + good_b)
14
15 P_A_given_D = (P_D_given_A * Pa) / ((P_D_given_A * Pa) + (P_D_given_B * Pb))
16
17 print("Probability that defective item came from Section A =", P_A_given_D)
18
```

The 'Run' window at the bottom shows the execution output:

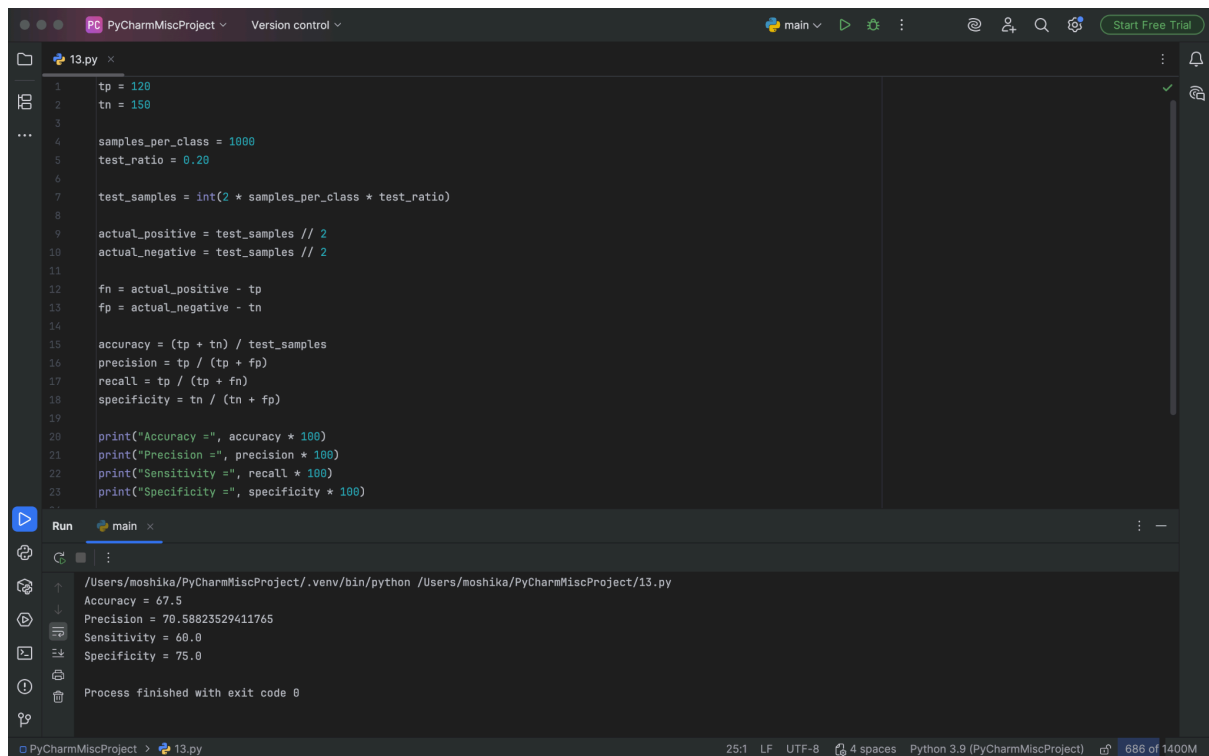
```
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/12.py
Probability that defective item came from Section A = 0.6428571428571428
Process finished with exit code 0
```

The status bar at the bottom indicates the file is '12.py', the encoding is 'UTF-8', and the Python version is 'Python 3.9 (PyCharmMiscProject)'.

13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. (5 Marks)

Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

ANSWER:



The screenshot shows a PyCharm IDE window with a Python script named 13.py. The script calculates various classification metrics based on the provided values: TP = 120, TN = 150, samples\_per\_class = 1000, and test\_ratio = 0.20. The script calculates test\_samples as 200, then actual\_positive as 100 and actual\_negative as 100. It then calculates fn, fp, accuracy, precision, recall, and specificity. The output of the script is displayed in the Run console, showing Accuracy = 67.5, Precision = 70.58823529411765, Sensitivity = 60.0, and Specificity = 75.0. The process finished with exit code 0.

```
1 tp = 120
2 tn = 150
3
4 samples_per_class = 1000
5 test_ratio = 0.20
6
7 test_samples = int(2 * samples_per_class * test_ratio)
8
9 actual_positive = test_samples // 2
10 actual_negative = test_samples // 2
11
12 fn = actual_positive - tp
13 fp = actual_negative - tn
14
15 accuracy = (tp + tn) / test_samples
16 precision = tp / (tp + fp)
17 recall = tp / (tp + fn)
18 specificity = tn / (tn + fp)
19
20 print("Accuracy =", accuracy * 100)
21 print("Precision =", precision * 100)
22 print("Sensitivity =", recall * 100)
23 print("Specificity =", specificity * 100)
```

Run main x

/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/13.py

Accuracy = 67.5  
Precision = 70.58823529411765  
Sensitivity = 60.0  
Specificity = 75.0

Process finished with exit code 0

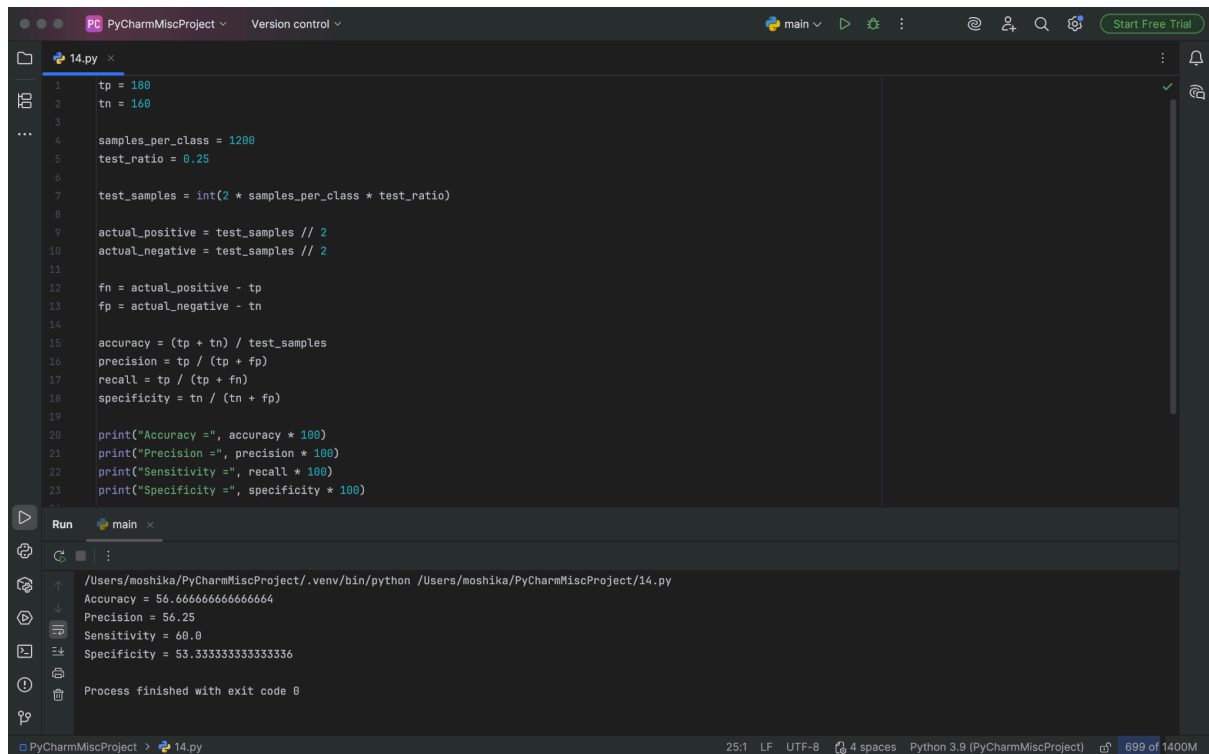
PyCharmMiscProject > 13.py 25.1 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 686 of 1400M

**14. A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data.**

**Calculate: Accuracy Precision Sensitivity Specificity.**

**(5 Marks)**

**ANSWER:**



The screenshot shows a PyCharm IDE window with a Python script named 14.py. The script calculates accuracy, precision, sensitivity, and specificity based on the provided values. The output of the script is displayed in the Run console.

```
1 tp = 180
2 tn = 160
3
4 samples_per_class = 1200
5 test_ratio = 0.25
6
7 test_samples = int(2 * samples_per_class * test_ratio)
8
9 actual_positive = test_samples // 2
10 actual_negative = test_samples // 2
11
12 fn = actual_positive - tp
13 fp = actual_negative - tn
14
15 accuracy = (tp + tn) / test_samples
16 precision = tp / (tp + fp)
17 recall = tp / (tp + fn)
18 specificity = tn / (tn + fp)
19
20 print("Accuracy =", accuracy * 100)
21 print("Precision =", precision * 100)
22 print("Sensitivity =", recall * 100)
23 print("Specificity =", specificity * 100)
```

Run console output:

```
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/14.py
Accuracy = 56.666666666666664
Precision = 56.25
Sensitivity = 60.0
Specificity = 53.333333333333336
Process finished with exit code 0
```

**15. A deep learning model is trained for a classification task, and the following loss values are observed:** (5 Marks)

- i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6**
- ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2**
- iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9**
- iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8**
- v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9**
- vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1**
- vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4**
- viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7**

**Answer the following:**

**a. At which epoch does the model achieve the best generalization performance?**

**ANSWER:**

The model achieves its best generalization performance at Epoch 4, because the validation loss is the lowest (1.8) at this epoch.

Answer: Epoch 4

**b. At which point does overfitting begin? Justify your answer.**

**ANSWER:**

Overfitting begins from Epoch 5 onwards.

Justification:

- The training loss continues to decrease from Epoch 5 to Epoch 8.
- The validation loss starts increasing after Epoch 4.
- This indicates that the model is fitting the training data too closely and its performance on unseen data is getting worse.

Answer: Overfitting begins at Epoch 5.

c. **Suggest two techniques to reduce overfitting and briefly explain how they help.**

**ANSWER:**

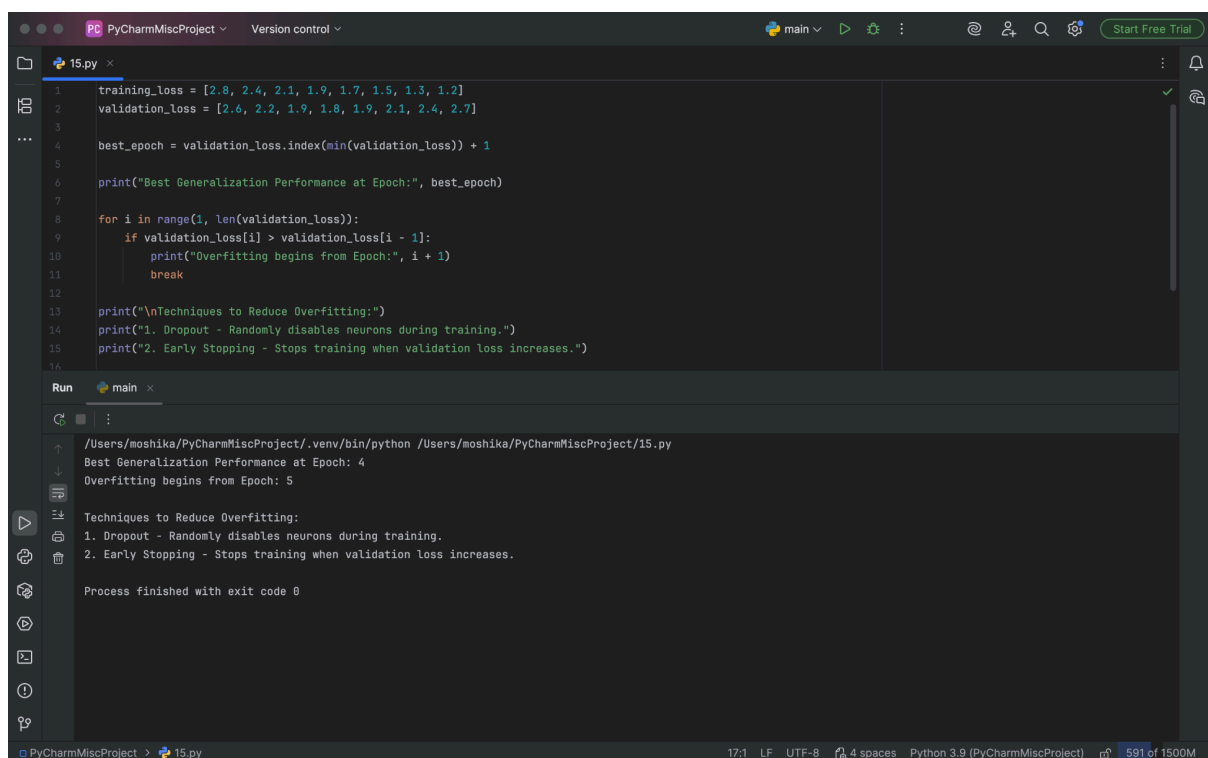
### 1. Dropout

- Randomly deactivates a fraction of neurons during training.
- Prevents the model from relying too much on specific neurons and improves generalization.

### 2. Early Stopping

- Stops the training process when the validation loss starts increasing.
- Prevents the model from learning unnecessary patterns or noise from the training data.

**FINAL OUTPUT:**



The screenshot displays the PyCharm IDE interface. The top toolbar includes icons for file operations, version control, and a 'Start Free Trial' button. The main editor window shows a Python script named '15.py' with the following code:

```
1 training_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
2 validation_loss = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]
3
4 best_epoch = validation_loss.index(min(validation_loss)) + 1
5
6 print("Best Generalization Performance at Epoch:", best_epoch)
7
8 for i in range(1, len(validation_loss)):
9     if validation_loss[i] > validation_loss[i - 1]:
10         print("Overfitting begins from Epoch:", i + 1)
11         break
12
13 print("\nTechniques to Reduce Overfitting:")
14 print("1. Dropout - Randomly disables neurons during training.")
15 print("2. Early Stopping - Stops training when validation loss increases.")
16
```

Below the editor, the 'Run' console shows the output of the script:

```
main
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/15.py
Best Generalization Performance at Epoch: 4
Overfitting begins from Epoch: 5

Techniques to Reduce Overfitting:
1. Dropout - Randomly disables neurons during training.
2. Early Stopping - Stops training when validation loss increases.

Process finished with exit code 0
```

The bottom status bar indicates the file is '15.py' in the 'PyCharmMiscProject' directory, using 'UTF-8' encoding with '4 spaces' for indentation, running on 'Python 3.9 (PyCharmMiscProject)'.